

Q.1) Write a C program to find whether a given file is present in current directory or not

[10 Marks]

```
#include <stdio.h>

#include <stdlib.h>

#include <dirent.h>

#include <string.h>


int main(int argc, char *argv[]) {

    if (argc != 2) {

        printf("Usage: %s <filename>\n", argv[0]);

        return 1;

    }


    char *filename = argv[1];

    DIR *dir = opendir(".");

    if (!dir) {

        perror("opendir");

        return 1;

    }


    int found = 0;

    struct dirent *entry;

    while ((entry = readdir(dir)) != NULL) {

        if (strcmp(entry->d_name, filename) == 0) {

            found = 1;

            break;

        }

    }


    closedir(dir);


    if (found)

        printf("File \"%s\" is present in current directory.\n", filename);
```

```
else

    printf("File \"%s\" is NOT present in current directory.\n", filename);

return 0;
}
```

Q.2) Write a C program to create an unnamed pipe. The child process will write following three messages to pipe and parent process display it.

```
Message1 = "Hello World"
Message2 = "Hello SPPU"
Message3 = "Linux is Funny" [20 Marks ]

#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <string.h>

int main() {
    int fd[2];
    pid_t pid;

    if (pipe(fd) == -1) {
        perror("pipe");
        exit(1);
    }

    pid = fork();
    if (pid < 0) {
        perror("fork");
        exit(1);
    }

    if (pid == 0) {
        // Child process: write messages to pipe
        close(fd[0]); // Close unused read end
```

```

char *messages[] = {
    "Hello World",
    "Hello SPPU",
    "Linux is Funny"
};

for (int i = 0; i < 3; i++) {
    write(fd[1], messages[i], strlen(messages[i]) + 1); // +1 to include '\0'
}

close(fd[1]);
exit(0);
} else {
    // Parent process: read messages from pipe
    close(fd[1]); // Close unused write end

    char buffer[100];
    int n;
    printf("Messages from child:\n");
    for (int i = 0; i < 3; i++) {
        n = read(fd[0], buffer, sizeof(buffer));
        if (n > 0)
            printf("%s\n", buffer);
    }

    close(fd[0]);
    wait(NULL); // Wait for child to terminate
}

return 0;
}

```